

Trabalho 2 — Árvore de Diretórios

Algoritmos e Estrutura de Dados III — Prof. Thiago Naves

Integrantes: Lucas Silva Maués (RA 2476878) · Leonardo José Reis Pinto (RA 2554097) · Rian Augusto de Matos Amaral (RA 2521717)

UTFPR — Campus Santa Helena

1. Objetivo e visão geral

O trabalho implementa, em linguagem C, um simulador de linha de comando que manipula diretórios e arquivos organizados em uma árvore. O programa lê um arquivo `in.txt` com a relação de pastas e arquivos, monta a árvore correspondente em memória e oferece um conjunto de comandos para navegar, buscar, criar e remover elementos, como faria um terminal de sistema operacional.

Diferentemente das árvores binárias e AVL vistas nas aulas anteriores, o problema pede uma **árvore genérica**: uma pasta pode conter qualquer número de subpastas e arquivos. Essa diferença foi o ponto de partida das nossas decisões de estrutura de dados, descritas nas seções 3 e 4.

O código está dividido em três arquivos, como exige o enunciado:

- `arvore.h` — definição das estruturas e declaração das funções;
- `arvore.c` — implementação das funções;
- `main.c` — função `main()`, com o laço da linha de comando, que apenas chama as funções implementadas.

Observação sobre o nome dos arquivos. O enunciado pede que os arquivos se chamem `matriz.h` e `matriz.c` e, na forma de avaliação, cita “novas funções com operações com matrizes”. Como este trabalho é de **árvores**, entendemos que esses termos provavelmente vieram do enunciado do Trabalho 1 (matriz esparsa) e não se aplicam diretamente aqui. Por coerência com o tema, optamos por nomear os arquivos `arvore.h` e `arvore.c`, mas respeitamos integralmente a divisão exigida: estrutura e declarações, implementação, e a `main()` separada. Caso a intenção fosse manter os nomes `matriz.h/matriz.c`, a renomeação é imediata e não altera o conteúdo.

2. Compilação e execução

O programa foi compilado e testado com o GCC, sem avisos, usando flags rígidas:

```
gcc -Wall -Wextra -std=c11 src/arvore.c src/main.c -o build/trabalho_arvores
./build/trabalho_arvores
```

Ao iniciar, o programa procura o arquivo de entrada (`in.txt`) em alguns locais usuais — o diretório atual, `src/` e a raiz do projeto — para funcionar tanto pela linha de comando quanto pelo botão de execução de uma IDE. Também é possível passar o caminho do arquivo como argumento: `./trabalho_arvores caminho/in.txt`. Se nenhum arquivo for encontrado, o programa inicia com a árvore vazia, e o usuário pode montá-la com `mkdir`.

3. Formato do arquivo de entrada

Cada linha do `in.txt` descreve um caminho completo, com os níveis separados por `/`, exatamente como no exemplo do enunciado:

```
Arquivos e Programas/Firefox
Arquivos e Programas/Chrome
Arquivos e Programas/Opera
Meus Documentos/apresentacao.ppt
Meus Documentos/relatorio.doc
Meus Documentos/fontes
Meus Documentos/fontes/main.c
Meus Documentos/fontes/main.h
Meus Documentos/imagens
Meus Downloads/7zip.exe
Meus Downloads/t2.rar
```

A distinção entre pasta e arquivo segue a regra dada no enunciado: **arquivo sempre tem extensão**. Assim, o último segmento de um caminho é tratado como arquivo se contiver um ponto (por exemplo `main.c`, `7zip.exe`) e como pasta caso contrário (por exemplo `Firefox`, `imagens`). Segmentos intermediários são sempre pastas, pois precisam conter outros elementos. Essa decisão está concentrada na função `tipoDoSegmento`, que apenas verifica a presença de `.` no segmento.

4. Estrutura de dados principal: árvore genérica

4.1 A escolha da representação

O ponto central do trabalho é como representar uma árvore em que cada nó pode ter um número arbitrário de filhos. A solução mais imediata seria guardar, em cada nó, um vetor (ou lista) de ponteiros para os filhos. Optamos por uma alternativa mais econômica e clássica na literatura: a representação **primeiro filho / próximo irmão** (em inglês, *left-child, right-sibling*).

Nessa representação, cada nó tem apenas três ponteiros, independentemente de quantos filhos possua:

```
typedef struct No {
    char nome[MAX_NOME];
```

```

TipoNo tipo;                /* PASTA ou ARQUIVO */
struct No *pai;             /* sobe na árvore (cd .., monta o caminho) */
struct No *primeiroFilho;   /* primeiro filho desta pasta */
struct No *proxIrmao;       /* próximo filho do mesmo pai */
} No;

```

A ideia é que os filhos de uma pasta formam uma lista encadeada simples: a pasta aponta para o primeiroFilho, e cada filho aponta para o proxIrmao seguinte, até o último, cujo proxIrmao é NULL. O ponteiro pai faz o caminho inverso, permitindo subir na árvore (comando `cd ..`) e reconstruir o caminho completo de qualquer nó.

4.2 Como o exemplo vira árvore

Conceitualmente, o `in.txt` acima corresponde a esta árvore de diretórios:

```

/
├── Arquivos e Programas/      (pasta)
│   ├── Firefox/              (pasta)
│   ├── Chrome/               (pasta)
│   └── Opera/                (pasta)
├── Meus Documentos/          (pasta)
│   ├── apresentacao.ppt      (arquivo)
│   ├── relatorio.doc         (arquivo)
│   ├── fontes/              (pasta)
│   │   ├── main.c           (arquivo)
│   │   └── main.h           (arquivo)
│   └── imagens/             (pasta)
└── Meus Downloads/           (pasta)
    ├── 7zip.exe              (arquivo)
    └── t2.rar                 (arquivo)

```

Internamente, porém, não existe “vetor de filhos”. O que existe são os ponteiros primeiroFilho e proxIrmao. A lista de filhos da raiz, por exemplo, é:

```

raiz "/"
  | primeiroFilho
  ▼
"Arquivos e Programas" —proxIrmao→ "Meus Documentos" —proxIrmao→ "Meus Downloads" → NULL

```

E cada uma dessas pastas tem, por sua vez, a sua própria lista de filhos ligada por proxIrmao (a seta ▼ indica o primeiroFilho):

```

"Arquivos e Programas"
  ▼
"Firefox"
  | proxIrmao
"Chrome"
  |
"Opera"

"Meus Documentos"
  ▼
"apresentacao.ppt"
  | proxIrmao
"relatorio.doc"
  |
"fontes" ──── "imagens"
              | primeiroFilho
              ▼
            "main.c" —proxIrmao→ "main.h"

"Meus Downloads"
  ▼
"7zip.exe"
  | proxIrmao
"t2.rar"

```

Para listar os filhos de uma pasta, basta começar em `primeiroFilho` e seguir `proxIrmao` até NULL. Para descobrir o pai de qualquer nó, basta ler o ponteiro `pai`. Essa é a operação que sustenta praticamente todos os comandos.

4.3 Por que essa representação

A motivação é o consumo de memória. Um vetor fixo de filhos teria de ser dimensionado para o pior caso (uma pasta com muitos filhos) e reservaria esse espaço em **todos** os nós, inclusive nas folhas, que não têm filho nenhum. A representação primeiro-filho/próximo-irmão usa sempre dois ponteiros para gerenciar os filhos, qualquer que seja o grau do nó. A análise quantitativa desse ganho está na seção 7.

Essa representação é a forma indicada na literatura para árvores de grau arbitrário. Cormen et al. (CLRS), na seção 10.4 (*Representing rooted trees*), descrevem exatamente essa técnica e observam que com ela cada nó ocupa espaço constante, o pai é alcançado em tempo constante e a lista de filhos é percorrida em tempo proporcional ao número de filhos. Ziviani, no *Projeto de Algoritmos*, também trata da representação de árvores por essa ideia de lista de filhos.

5. Estrutura auxiliar: índice de busca por Trie

5.1 O problema que o índice resolve

O comando `search` precisa encontrar um arquivo ou pasta pelo nome e informar sua localização. A forma ingênua seria, a cada busca, percorrer a árvore inteira comparando o nome de cada nó — um custo $O(N)$, sendo N o total de nós. Para evitar isso, mantivemos um **índice** paralelo à árvore.

Essa ideia de índice foi sugerida em sala pelo professor, inspirada no que foi feito no Trabalho 1 (a matriz esparsa, que mantém uma estrutura própria para acessar rapidamente apenas os elementos relevantes). Aqui, o índice é uma **Trie**, também chamada de árvore digital.

5.2 Como a Trie funciona

A Trie é uma árvore em que cada aresta corresponde a um caractere do nome. Para inserir um nome, descemos a partir da raiz da Trie seguindo um filho por caractere, criando os nós que faltarem. O nó onde o nome termina guarda a informação de que aquele nome existe. Como nomes podem se repetir na árvore de diretórios (vários `main.c`, por exemplo), cada ponto de término guarda uma **lista encadeada de ocorrências**, isto é, a lista de todos os nós da árvore que têm aquele nome:

```
typedef struct OcorrenciaNo { /* lista de nós que têm um certo nome */
    No *no;
    struct OcorrenciaNo *prox;
} OcorrenciaNo;

typedef struct TrieNo { /* um filho por byte possível */
```

```

struct TrieNo *filhos[ALFABETO]; /* ALFABETO = 256 */
OcorrenciaNo *ocorrencias;      /* nós cujo nome termina aqui */
} TrieNo;

```

Usamos um alfabeto de 256 (um filho por byte possível) justamente porque os nomes do enunciado contêm espaços e podem conter acentos; tratar o nome byte a byte evita ter de mapear caracteres especiais. Inserir `main.c` e `main.h`, que compartilham o prefixo `main.`, produz:

```

(raiz da Trie)
├─ m ─ a ─ i ─ n ─ . ─┬─ c → ocorrência: nó "main.c"
                     └─ h → ocorrência: nó "main.h"

```

5.3 O ganho de desempenho

Com a Trie, a busca exata por um nome custa $O(|nome|)$ — proporcional ao **comprimento do nome**, e não ao número de nós da árvore. Descemos um nível por caractere e, ao chegar ao fim, devolvemos a lista de ocorrências.

A Trie também oferece, de graça, a **busca por prefixo** exigida pelo comando `cd`. Quando o nome buscado não existe exatamente, descemos até onde o prefixo levar e percorremos a subárvore abaixo desse ponto, coletando todos os nomes que começam pelo termo digitado. É o que permite que `search Me` ou `cd Me` sugiram “Meus Documentos” e “Meus Downloads”.

O índice precisa ser mantido em sincronia com a árvore: toda criação de nó (na carga do arquivo e no `mkdir`) registra o nó na Trie, e toda remoção (`rm`) retira o nó da Trie antes de liberá-lo. Esse cuidado é o que evita que o `search` aponte para memória já liberada.

6. Os comandos

A construção da árvore e cada comando estão implementados como funções próprias, chamadas pelo laço principal em `main.c`. A seguir, descrevemos o funcionamento de cada um.

Carga do arquivo (`lerArquivo` e `inserirCaminho`). A função `lerArquivo` abre o `in.txt` e lê uma linha por vez. Cada linha é entregue a `inserirCaminho`, que a quebra em segmentos com `strtok` usando `/` como separador. Caminhando da raiz para baixo, para cada segmento o programa verifica se já existe um filho com aquele nome (`buscarFilho`); se não existir, cria o nó, liga-o ao pai (`adicionarFilho`) e o registra no índice. Assim, linhas que compartilham um prefixo (como as três que começam por `Arquivos` e `Programas`) reaproveitam os nós já criados, em vez de duplicá-los.

`cd <diretório>` — entrar em uma pasta. Trata primeiro os casos especiais `.` (permanece), `/` (volta à raiz) e `..` (sobe para o pai, usando o ponteiro `pai`). Em seguida procura, entre os filhos da pasta atual, um com o nome exato. Se encontrar e for pasta, entra nela; se for arquivo, avisa que não é um diretório. Se não encontrar, cumpre a exigência do enunciado: percorre os filhos da pasta atual e imprime os nomes de pasta que **começam** com o texto digitado, como sugestões; se não houver nenhuma sugestão, imprime “Diretório não encontrado”. A busca do

`cd` é feita localmente entre os filhos da pasta corrente (e não pelo índice global) porque `cd` é relativo ao diretório atual: faz sentido sugerir apenas o que está logo abaixo dele.

search <arg> — localizar arquivo ou pasta. Consulta o índice. Primeiro tenta o casamento exato pela Trie, em $O(|arg|)$; havendo ocorrências, imprime o caminho completo de cada uma (reconstruído pelos ponteiros `pai`), indicando se é pasta ou arquivo. Se não houver casamento exato, usa a busca por prefixo da Trie e lista, como sugestão, tudo o que começa com o termo. Se nem isso retornar nada, informa que nada foi encontrado.

rm <diretório> — remover com liberação recursiva. Localiza o alvo entre os filhos da pasta atual. Primeiro desliga o nó da lista de irmãos do pai (`desligarDoPai`), religando os ponteiros para que a lista continue íntegra. Em seguida libera toda a subárvore com `liberarSubarvore`, que é **recursiva**: para cada nó, libera antes todos os seus filhos (percorrendo `primeiroFilho` / `proxIrmao`) e só então libera o próprio nó, em pós-ordem. Antes de liberar cada nó, ele é retirado do índice, mantendo a Trie coerente. Isso atende diretamente à exigência do enunciado de “liberação recursiva”.

list — listar o conteúdo da pasta atual. Percorre os filhos da pasta corrente seguindo `primeiroFilho` e `proxIrmao`, imprimindo cada nome. As pastas são marcadas com uma barra final (`fontes/`), o que as distingue visualmente dos arquivos. Se a pasta não tiver filhos, informa que está vazia.

mkdir <arg> — criar pasta. Verifica se já existe um elemento com aquele nome na pasta atual; se existir, avisa e não faz nada. Caso contrário, cria um nó do tipo pasta, liga-o à pasta atual e o registra no índice, de modo que a nova pasta já pode ser encontrada pelo `search` e usada pelo `cd`.

clear — limpar a tela. Usa a chamada de sistema apropriada ao sistema operacional, escolhida em tempo de compilação: `cls` no Windows e `clear` no Linux/macOS. Caso a chamada falhe (por exemplo, em um ambiente sem esse comando), recorre ao plano B previsto no enunciado: imprime várias linhas em branco. Assim, o comando funciona nos dois sistemas citados pelo enunciado.

help — ajuda. Como pede o enunciado, é um comando personalizado que lista todos os comandos disponíveis, o modo de uso e a finalidade de cada um.

exit — encerrar liberando a memória. Antes de terminar, o programa libera **toda** a memória alocada: a árvore inteira (`liberarArvore`, também recursiva e em pós-ordem) e o índice (`liberarTrie`, que libera os nós da Trie e as listas de ocorrências). Só depois encerra.

mem — comparativo de memória (extra). Comando adicional, descrito na seção 7, criado para demonstrar quantitativamente o ganho da representação escolhida.

Para ilustrar, esta é uma sessão real do programa reproduzindo o exemplo do enunciado (o caso `cd Me`):

```
/> cd Me
Diretorio 'Me' nao encontrado. Voce quis dizer:
Meus Documentos
```

Meus Downloads

```
/> cd Meus Documentos  
/Meus Documentos> search main.c  
Encontrado:  
[arquivo] /Meus Documentos/fontes/main.c
```

7. Análise de memória

Como elemento adicional, o comando `mem` mede, sobre a árvore efetivamente carregada, o custo de memória da representação escolhida em comparação com a alternativa de vetor fixo de filhos. Procuramos apresentar o resultado de forma honesta, separando dois recortes, para não superestimar o ganho.

O **primeiro recorte** é a parte que a escolha de representação realmente controla: o gerenciamento dos filhos. A representação primeiro-filho/ próximo-irmão usa 2 ponteiros por nó (`primeiroFilho` e `proxIrmão`); um vetor fixo dimensionado para um grau máximo de 50 usaria 50 ponteiros mais um contador. Nesse recorte, a diferença é de cerca de **16 bytes por nó contra 404 bytes por nó — aproximadamente 25 vezes menos**.

O **segundo recorte** é a estrutura completa do nó. Aqui é preciso reconhecer que o campo `nome[256]` é idêntico nas duas alternativas e domina o tamanho do nó. Por isso, quando se compara a struct inteira, a vantagem cai para cerca de **2,3 vezes** (288 bytes contra 676 bytes por nó, na árvore de exemplo). Mostramos os dois números de propósito: o ganho da representação é real e cresce à medida que o grau máximo aumenta, mas fica diluído pelo nome de tamanho fixo. Apresentar apenas o número de 25 vezes, sem essa ressalva, daria uma impressão exagerada do resultado.

O índice (Trie) tem custo de memória próprio e relativamente alto, porque cada nó da Trie reserva 256 ponteiros de filhos. Ele troca memória por velocidade de busca: o `search` passa de $O(N)$ para $O(|nome|)$. Esse é exatamente o compromisso que motiva a **Patricia** (Ziviani, seção 5.4.2), uma evolução da Trie que comprime os nós de passagem com um único filho e reduz bastante esse consumo. Não a implementamos, mas a citamos como o caminho natural de otimização do índice.

8. Desafios e dificuldades

O primeiro obstáculo foi conceitual: entender que “árvore genérica” não é árvore binária. As aulas anteriores trataram de árvores binárias e AVL, em que cada nó tem no máximo dois filhos. Aqui foi preciso buscar a representação primeiro-filho/próximo-irmão para permitir um número arbitrário de filhos sem desperdiçar memória.

A reconstrução do caminho completo de um nó (usada no `search` e no `prompt`) foi resolvida com o ponteiro `pai`: empilhamos os nós da posição atual até a raiz e os imprimimos de cima para baixo.

O tratamento de nomes com espaço, como `cd Meus Documentos`, exigiu um cuidado no *parser* da linha digitada: separamos apenas o primeiro espaço (que divide o comando do argumento) e tomamos todo o resto da linha como o argumento, em vez de quebrar em cada espaço.

Manter o índice coerente na remoção foi o ponto mais delicado. Ao liberar uma subárvore, cada nó precisa ser retirado da Trie **antes** de ser liberado; caso contrário, uma busca posterior poderia seguir um ponteiro para memória já liberada. Por isso a liberação é feita em pós-ordem, retirando o nó do índice no momento certo.

Por fim, a própria medição de memória foi um aprendizado. A primeira estimativa, feita “de cabeça”, exagerava o ganho. Ao medir de fato, percebemos que o nome [256] dilui a diferença, o que nos levou a apresentar os dois números (o ganho no gerenciamento de filhos e o ganho na struct completa) em vez de um só.

9. Verificação

- Compilação sem avisos com `gcc -Wall -Wextra -std=c11`.
- Todos os comandos testados com a árvore de exemplo do enunciado, incluindo o caso `cd Me` com as sugestões “Meus Documentos” e “Meus Downloads”.
- Ausência de vazamentos de memória: após exercitar `rm` e `exit`, a ferramenta `leaks` reportou “*0 leaks for 0 total leaked bytes*”.

10. Referências

1. CORMEN, T. H.; LEISERSON, C. E.; RIVEST, R. L.; STEIN, C. *Introduction to Algorithms*. Seção 10.4 — *Representing rooted trees* (representação primeiro-filho/próximo-irmão).
2. ZIVIANI, N. *Projeto de Algoritmos com Implementações em Pascal e C*. Representação de árvores e seção 5.4 — Pesquisa Digital (5.4.1 Trie; 5.4.2 Patricia).